Լաբորատոր աշխատանք 6
Բանկային հաշիվների և փոխանցումների համակարգ

Աշխատանքի նպատակը. Ուսումնասիրել և գործնականում կիրառել C++ լեզվում դասերի (class) ստեղծման սկզբունքները, մասնավորապես՝ private դաշտերի, public կոնստրուկտորների և մեթոդների կիրառությունը: Սովորել օբյեկտների զանգվածների (array of objects) ստեղծումն ու կառավարումը: Ապահովել մուտքային տվյալների խիստ վավերացում և ստուգումներ դասի մեթոդների միջոցով: Զարգացնել օբյեկտ կողմնորոշված ծրագրավորման (OOP) հմտությունները՝ իրականացնելով տվյալների մշակում, որոնում, վիճակագրության հաշվարկ և տեսակավորում հաշվետվությունների համար:
Խնդրի պահանջը. Յուրաքանչյուր խնդրի համար անհրաժեշտ է սահմանել համապատասխան class՝ մասնավոր (private) և հանրային (public) բաժիններով: Ստեղծել BankAccount դասը հետևյալ դաշտերով՝ accountNumber, owner, balance, dailyLimit, blocked: Ստեղծել 10 հաշիվների զանգված և իրականացնել փոխանցումներ օբյեկտների միջև: Ապահովել մուտքային տվյալների ամբողջական ստուգում (այդ թվում՝ սկզբնական մնացորդի և օրական սահմանաչափի համեմատություն, ինչպեսև կրկնվող հաշվեհամարների բացառում):
Առաջադրանք. Ծրագրի միջոցով հարկավոր է իրականացնել հետևյալ գործողությունները.
Կոնստրուկտորի միջոցով ստուգում. Ստուգել սկզբնական մնացորդի և օրական սահմանաչափի վավերականությունը (մնացորդը չպետք է լինի բացասական և չպետք է փոքր լինի օրական սահմանաչափից): Գումարի մուտքագրում և ելքագրում (deposit() և withdraw() մեթոդներ). Փոխել մնացորդը՝ խստիվ արգելելով բացասական գումարների մուտքը, արգելափակված հաշիվներով գործարքները և սահմանաչափի ու հասանելի միջոցների գերազանցումը: Փոխանցումներ (transferTo() մեթոդ). Փոխանցել գումար մի հաշվից մյուսին՝ նախապես ստուգելով երկու հաշիվների կարգավիճակն ու սահմանաչափերը: Վիճակագրություն. Հաշվել հաջողված փոխանցումների ընդհանուր ծավալը և գտնել ամենաակտիվ հաշիվը: Հաշվետվություն. Կազմել ամփոփ հաշվետվություն՝ հաշիվները դասավորելով ըստ մնացորդի նվազման կարգի և հստակ առանձնացնելով ակտիվ ու արգելափակված հաշիվները:

```
#include <iostream>
#include <string>
using namespace std;
class BankAccount {
private:
    string accountNumber;
    string owner;
    double balance;
    double dailyLimit;
    bool blocked;
    int successfulTransfersCount;
public:
    BankAccount() {
```

```
    accountNumber = "";
    owner = "";
    balance = 0.0;
    dailyLimit = 0.0;
    blocked = false;
    successfulTransfersCount = 0;
}
void inputAccount(int index, const BankAccount accounts[]) {
    cout << "\n--- Enter account details #" << index + 1 << " ---" << endl;
    bool exists;
    do {
        exists = false;
        cout << "Account number (e.g., ACC001): ";
        cin >> accountNumber;
        if (accountNumber.empty()) {
            cout << "Error: Account number cannot be empty! Please try again:" << endl;
            exists = true;
            continue;
        }
        for (int i = 0; i < index; ++i) {
            if (accounts[i].getAccountNumber() == accountNumber) {
                cout << "Error: Account number already exists! Please enter a different number:" << endl;
                exists = true;
                break;
            }
        }
    } while (exists);

    do {
        cout << "Account owner (name): ";
        cin >> owner;
        if (owner.empty()) {
            cout << "Error: Owner name cannot be empty! Please try again:" << endl;
        }
    } while (owner.empty());

    bool validBalanceAndLimit = false;
    do {
        do {
            cout << "Account balance (>= 0): ";
            cin >> balance;
            if (cin.fail() || balance < 0) {
                cin.clear();
                cin.ignore(10000, '\n');
                cout << "Error: Invalid balance! Please enter a non-negative value:" << endl;
            }
        } while (balance < 0);

        do {
            cout << "Daily limit (>= 0): ";
            cin >> dailyLimit;
            if (cin.fail() || dailyLimit < 0) {
                cin.clear();
                cin.ignore(10000, '\n');
                cout << "Error: Invalid daily limit! Please enter a non-negative value:" << endl;
            }
        } while (dailyLimit < 0);
```

```
            if (balance < dailyLimit) {
                cout << "Error: Account balance cannot be less than the daily limit! Please enter a different value:" <<
endl;
            }
            else {
                validBalanceAndLimit = true;
            }
        } while (!validBalanceAndLimit);

        int status;
        do {
            cout << "Status (0 - Active, 1 - Blocked): ";
            cin >> status;
            if (cin.fail() || (status != 0 && status != 1)) {
                cin.clear();
                cin.ignore(10000, '\n');
                cout << "Error: Please enter either 0 or 1!" << endl;
            }
        } while (status != 0 && status != 1);

        blocked = (status == 1);
        successfulTransfersCount = 0;
    }
    void deposit(double amount) {
        if (blocked) {
            cout << "Error: Account (" << accountNumber << ") is blocked!" << endl;
            return;
        }
        if (amount <= 0) {
            cout << "Error: Invalid deposit amount! Please enter a positive value:" << endl;
            return;
        }
        balance += amount;
        cout << "Deposit: " << amount << " AMD deposited into account " << accountNumber << "." << endl;
    }
    bool withdraw(double amount) {
        if (blocked) {
            cout << "Error: Account (" << accountNumber << ") is blocked!" << endl;
            return false;
        }
        if (amount <= 0) {
            cout << "Error: Invalid withdrawal amount! Please enter a positive value:" << endl;
            return false;
        }
        if (amount > balance) {
            cout << "Error: Insufficient funds in account (" << accountNumber << ")!" << endl;
            return false;
        }
        if (amount > dailyLimit) {
            cout << "Error: Withdrawal amount exceeds the daily limit (" << dailyLimit << ")!" << endl;
            return false;
        }
        balance -= amount;
        cout << "Withdrawal: " << amount << " AMD withdrawn from account " << accountNumber << "." << endl;
        return true;
    }
    bool transferTo(BankAccount& recipient, double amount, double& totalVolume) {
        if (this->accountNumber == recipient.accountNumber) {
```

```
            cout << "Error: Cannot transfer to the same account!" << endl;
            return false;
        }
        if (blocked) {
            cout << "Error: Account (" << accountNumber << ") is blocked!" << endl;
            return false;
        }
        if (recipient.blocked) {
            cout << "Error: Recipient account (" << recipient.accountNumber << ") is blocked!" << endl;
            return false;
        }
        if (amount <= 0) {
            cout << "Error: Invalid transfer amount! Please enter a positive value:" << endl;
            return false;
        }
        if (amount > balance) {
            cout << "Error: Insufficient funds in account (" << accountNumber << ")!" << endl;
            return false;
        }
        if (amount > dailyLimit) {
            cout << "Error: Transfer amount exceeds the daily limit (" << dailyLimit << ")!" << endl;
            return false;
        }
        balance -= amount;
        recipient.balance += amount;

        successfulTransfersCount++;
        totalVolume += amount;

        cout << "Transfer: " << amount << " AMD transferred from account " << accountNumber << " to account " <<
recipient.accountNumber << "." << endl;
        return true;
    }
    string getAccountNumber() const { return accountNumber; }
    string getOwner() const { return owner; }
    double getBalance() const { return balance; }
    double getDailyLimit() const { return dailyLimit; }
    bool isBlocked() const { return blocked; }
    int getTransfersCount() const { return successfulTransfersCount; }
    void display() const {
        cout << "Account: " << accountNumber
            << " | Owner: " << owner
            << " | Balance: " << balance
            << " | Limit: " << dailyLimit
            << " | Status: " << (blocked ? "Blocked" : "Active") << endl;
    }
};
void swapAccounts(BankAccount& a, BankAccount& b) {
    BankAccount temp = a;
    a = b;
    b = temp;
}
int main() {
    const int SIZE = 10;
    BankAccount accounts[SIZE];
    cout << "=== INPUT DATA FOR 10 ACCOUNTS ===" << endl;
    for (int i = 0; i < SIZE; ++i) {
        accounts[i].inputAccount(i, accounts);
```

```
}
cout << "\n=== ALL ACCOUNTS ===" << endl;
for (int i = 0; i < SIZE; ++i) {
    accounts[i].display();
}
double totalSuccessfulVolume = 0.0;
char choice;

cout << "\n=== TRANSFER PROCESS ===" << endl;
do {
    string senderAcc, receiverAcc;
    double amount;
    cout << "\nSender account: ";
    cin >> senderAcc;
    cout << "Recipient account: ";
    cin >> receiverAcc;
    do {
        cout << "Transfer amount: ";
        cin >> amount;
        if (cin.fail() || amount <= 0) {
            cin.clear();
            cin.ignore(10000, '\n');
            cout << "Error: Invalid amount! Please try again:" << endl;
        }
    } while (amount <= 0);
    int senderIdx = -1, receiverIdx = -1;
    for (int i = 0; i < SIZE; ++i) {
        if (accounts[i].getAccountNumber() == senderAcc) senderIdx = i;
        if (accounts[i].getAccountNumber() == receiverAcc) receiverIdx = i;
    }
    if (senderIdx == -1) {
        cout << "Error: Sender account (" << senderAcc << ") not found!" << endl;
    }
    else if (receiverIdx == -1) {
        cout << "Error: Recipient account (" << receiverAcc << ") not found!" << endl;
    }
    else {
        accounts[senderIdx].transferTo(accounts[receiverIdx], amount, totalSuccessfulVolume);
    }
    cout << "Perform another transfer? (y/n): ";
    cin >> choice;
} while (choice == 'y' || choice == 'Y');
cout << "\n=== STATISTICS ===" << endl;
cout << "Total volume of successful transfers: " << totalSuccessfulVolume << endl;
int mostActiveIdx = 0;
for (int i = 1; i < SIZE; ++i) {
    if (accounts[i].getTransfersCount() > accounts[mostActiveIdx].getTransfersCount()) {
        mostActiveIdx = i;
    }
}
cout << "Most active account: " << accounts[mostActiveIdx].getAccountNumber()
    << " (" << accounts[mostActiveIdx].getOwner() << ") - Transfers: "
    << accounts[mostActiveIdx].getTransfersCount() << endl;
for (int i = 0; i < SIZE - 1; ++i) {
    for (int j = 0; j < SIZE - i - 1; ++j) {
        if (accounts[j].getBalance() < accounts[j + 1].getBalance()) {
            swapAccounts(accounts[j], accounts[j + 1]);
        }
```

```
        }
    }
    cout << "\n=== REPORT (Sorted by balance in descending order) ===" << endl;
    cout << "\n--- ACTIVE ACCOUNTS ---" << endl;
    for (int i = 0; i < SIZE; ++i) {
        if (!accounts[i].isBlocked()) {
            accounts[i].display();
        }
    }
   cout << "\n--- BLOCKED ACCOUNTS ---" << endl;
    for (int i = 0; i < SIZE; ++i) {
        if (accounts[i].isBlocked()) {
            accounts[i].display();
        }
    }
    return 0;
}
```